EMTP: Efficient Multi-Robot Task Planning with Large Language Models

1st Seungin Park
Dept. Mechanical Engineering
UNIST
Ulsan, Republic of Korea
qkrwk0921@unist.ac.kr

2st Minjae Jung
Dept. Mechanical Engineering
KAIST
Daejeon, Republic of Korea
starshirts@kaist.ac.kr

3st Changseung Kim
Dept. Mechanical Engineering
UNIST
Ulsan, Republic of Korea
pon02124@unist.ac.kr

4st Woojae Shin
Dept. Mechanical Engineering
KAIST
Daejeon, Republic of Korea
oj7987@kaist.ac.kr

5st Seunghoon Lee
Dept. Mechanical Engineering
KAIST
Daejeon, Republic of Korea
lsh356812@kaist.ac.kr

6st Hyondong Oh
Dept. Mechanical Engineering
KAIST
Daejeon, Republic of Korea
h.oh@kaist.ac.kr

***Abstract*—Dynamic task allocation in heterogeneous multi-robot systems requires efficient replanning under evolving environmental conditions and operator instructions. While conventional methods often rely on predefined models, existing LLM-based approaches face challenges such as increasing reasoning cost, hallucinations, and limited scalability when handling multiple tasks and dynamic constraints. To address these limitations, we propose EMTP, an efficient multi-robot task planning framework that integrates 3D Scene Graph (3DSG)-based grounding with supervisor-based hierarchical LLM orchestration. EMTP selectively invokes semantic filtering, semantic reasoning, and optimization-based task allocation agents according to the type of dynamic event, reducing unnecessary reasoning overhead while supporting constraint-aware planning. The 3DSG provides structured environmental context for both LLM-based reasoning and optimization-based allocation. Simulation experiments in large-scale indoor environments demonstrate that EMTP achieves higher task completion and constraint satisfaction rates with lower planning computation time and token usage than existing LLM-based multi-robot planning methods, particularly in online replanning scenarios with accumulated dynamic constraints.**

***Index Terms*—Multi-robot task allocation, dynamic replanning, large language models**

I. INTRODUCTION

MULTI-robot systems have shown significant potential in performing diverse tasks in parallel across complex environments, with applications in various domains, such as logistics, inspection, and search-and-rescue operations. In real-world environments, dynamic situations such as the emergence of obstacles, changes in task states, and additional commands from operators frequently occur, requiring continuous task reallocation and path replanning. In particular, environments that involve multiple tasks and dynamic changes significantly increase the complexity of the problem, requiring efficient task allocation that simultaneously satisfies the heterogeneity of the robot, the feasibility of the task and the consistency between the constraints of the task.

Conventional optimization-based and rule-based methods have shown effective performance in structured environments by focusing on optimal task assignment and cost minimization. However, these methods typically rely on predefined environmental models, which limits their ability to respond to dynamic situations and incorporate complex semantic constraints. Learning-based methods can provide data-driven flexibility, but they still face limitations in terms of robustness to environmental changes and the interpretability of the resulting plans.

To address these limitations, recent studies have explored the integration of large language models (LLMs) into multi-robot planning by leveraging their natural language understanding and high-level reasoning capabilities. However, existing studies have focused mainly on task decomposition and plan generation, while task allocation in scenarios involving multiple tasks and continuous environmental changes remains insufficiently investigated. Moreover, as the amount of input information increases, LLMs suffer from rapidly increasing inference time and computational cost, and their outputs may lose consistency and reliability due to hallucinations. These issues become more severe when multiple tasks and environmental information must be processed simultaneously, limiting the direct use of LLMs as online planners in dynamic environments. Therefore, a scalable task allocation architecture is needed to efficiently and reliably handle multiple tasks in large-scale indoor environments.

To address these challenges, this paper proposes EMTP, an efficient dynamic task allocation framework for heterogeneous multi-robot systems in large-scale indoor environments. The main contributions of this work are summarized as follows:

This work was supported by the Unmanned Vehicles Core Technology Research and Development Program through the National Research Foundation of Korea (NRF) and Unmanned Vehicle Advanced Research Center (UVARC) funded by the Ministry of Science and ICT, the Republic of Korea (2020M3C1C1A01082375), and by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (2023R1A2C2003130).

- We propose an LLM-based heterogeneous multi-robot task allocation framework for efficient operation under multiple tasks and dynamic environmental conditions.
- We introduce hierarchical adaptive LLM orchestration and a 3D scene graph-based dynamic environment representation to reduce planning cost and latency, enabling reliable online planning.
- We validate through extensive experiments in large-scale indoor environments that the proposed framework outperforms existing methods in terms of task completion rate and planning efficiency.

II. RELATED WORKS

A. LLM-based Task Planning

Research on LLM-based task planning focuses on generating feasible plans along two main approaches: integrating LLMs with formal methods and grounding them in structured scene representations. Classical formal languages, such as Linear Temporal Logic (LTL) and the Planning Domain Definition Language (PDDL), enable the generation of executable plans from natural language instructions while satisfying temporal and logical constraints among tasks [1]–[3]. However, these approaches depend on predefined symbolic representations, which require substantial domain expertise. While effective in static and small-scale environments, they have limited scalability and adaptability in large-scale and dynamic environments.

Other studies ground LLMs in structured environmental information, such as 3D Scene Graph (3DSG), to support semantic reasoning over object and spatial relationships for long-horizon task planning [4], [5]. SayPlan [4] collapses a static 3DSG into subgraphs and performs semantic search over them to find task-relevant objects and spatial relationships. MoMa-LLM [5] dynamically constructs a local scene graph during exploration in unmapped environments and performs object search based on it. However, these studies primarily focus on single-robot scenarios and do not adequately address multi-robot challenges such as task allocation, coordination, and cooperation across multiple tasks.

B. Conventional Multi-Robot Task Planning

Multi-robot task planning consists of task decomposition and task allocation [6]. Motes et al. [7] showed through TMP-CBS that task decomposition, task allocation, and motion planning should be considered in an integrated manner. However, most conventional studies assume predefined executable subtasks and mainly focus on allocation efficiency and optimality.

Optimization-based approaches formulate task allocation and path planning as combinatorial optimization problems, such as the multiple Traveling Salesman Problem (mTSP) [8], [9], or dynamically integrate scheduling with motion planning [7], [10], [11]. Although these methods perform well in structured environments, they remain limited in dynamic environments, particularly when environmental changes occur, such as unexpected obstacles or changes in task states.

Consensus-based methods address allocation in a decentralized manner through distributed auctions [12], [13]. However, the convergence rate often degrades in large-scale dynamic environments, and it remains difficult to handle complex task constraints among heterogeneous robots.

Learning-based methods have improved flexibility and scalability [14]–[20]. However, these approaches remain vulnerable to unseen environments and fail to generate explainable plans, making it difficult to consistently incorporate operator instructions into generated plans.

C. LLM-based Multi-Robot Allocation and Planning

To address the limitations of conventional multi-robot task planning, recent studies have applied LLMs to multi-robot collaborative planning and task allocation, leveraging their natural language understanding and high-level reasoning capabilities. SMART-LLM [21] and RoCo [22] showed that LLMs can serve as high-level reasoning modules for multi-robot cooperative planning and coordination through a staged task-planning framework and a dialogue-based collaboration framework, respectively. LaMMA-P [23] and LiP-LLM [24] aimed to improve the structural consistency and feasibility of multi-robot plans by integrating LLMs with PDDL-based formulations and optimization-based methods. These studies indicate that LLMs can serve not only as natural language interpreters but also as decision-making modules for task decomposition, multi-robot coordination, and constraint-aware planning.

However, existing approaches are largely confined to scenarios with a limited number of robots and tasks, often evaluated within static or simplified environments. In dialogue-based planning, maintaining an increasingly long interaction history poses a major scalability challenge. As the number of robots and interactions increases, the prompt size grows rapidly, limiting the ability of LLMs to reason consistently within a bounded context window. While combining LLMs with formal representations or optimization techniques improves plan feasibility and structural consistency, these approaches often require LLMs to produce structured plans or dependency graphs beyond high-level reasoning. This reliance on structured generation may introduce risks such as hallucination and logical inconsistencies. Moreover, constraining the output to predefined formal structures can limit their ability to accommodate diverse and unexpected constraint types, such as spatial restrictions or robot failures. These limitations make it difficult to support continuous task reallocation and replanning in large-scale dynamic environments.

COHERENT [25] mitigates some of these limitations by introducing a centralized Plan-Execute-Feedback-Adapt (PEFA) architecture for long-horizon heterogeneous robot collaboration and closed-loop error recovery, showing that LLM-based multi-robot planning can extend beyond one-shot plan generation to include feedback and recovery during execution. However, its iterative feedback-based structure incurs increasing computational cost and latency as the number of planning steps grows, making it difficult to maintain both efficiency and consistency in dynamic environments with multiple concurrent tasks. Consequently, a scalable task allocation architecture

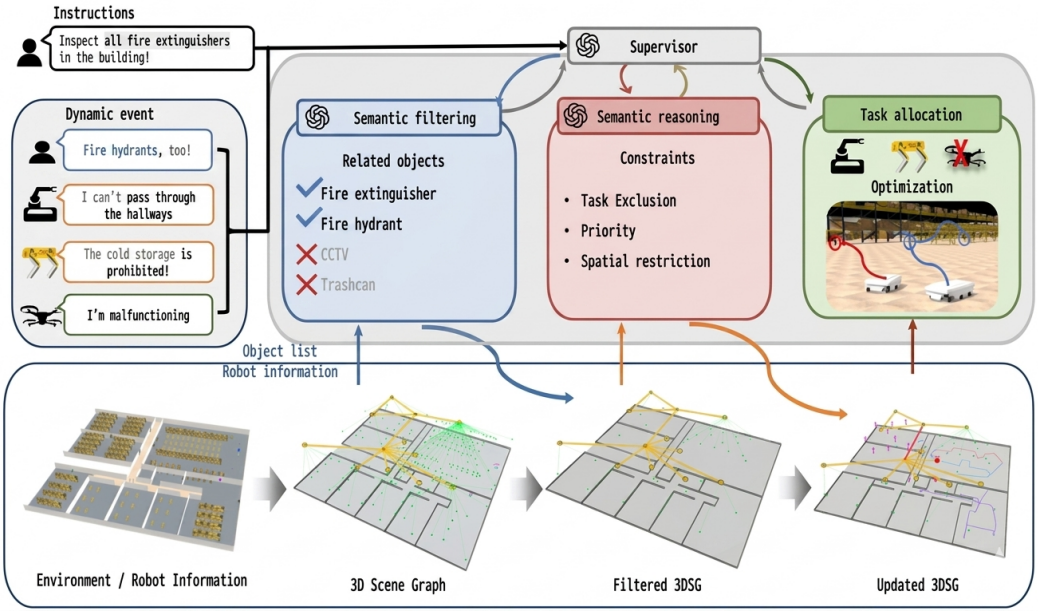


Fig. 1. Overview of the EMTP framework. The Supervisor agent coordinates semantic filtering, semantic reasoning, and optimization-based task allocation using operator instructions, dynamic events, and 3DSG-based environmental information. When a dynamic event occurs, the replanning pipeline is selectively re-executed according to the event type.

is essential to reliably handle multiple robots and tasks in large-scale environments. Such an architecture should adapt in real time to dynamic task conditions and environmental uncertainties, enabling timely task reallocation and continuous replanning.

III. PROPOSED METHOD

As shown in Fig. 1, EMTP performs task allocation and planning for heterogeneous multi-robot systems through supervisor-based hierarchical orchestration. We consider a large-scale indoor environment consisting of multiple floors and rooms, where three types of heterogeneous robots—unmanned aerial vehicles (UAVs), unmanned ground vehicles (UGVs), and quadruped robots—are deployed. Given a set of task instructions $\mathcal{I} = \{i_1, i_2, \ldots, i_n\}$, where i_1: "Inspect all fire extinguishers in the building," tasks are defined as visits to multiple waypoints. During execution, two types of dynamic events may occur: physical constraint events arising from the environment, such as obstacle emergence and robot malfunctions, and semantic constraint events arising from operator instructions, such as task priority changes, task exclusions, and restrictions on specific areas.

The Supervisor LLM agent takes as input the operator's instructions, dynamic events, and 3DSG-based environmental information (Section III-A), and coordinates the entire task allocation process. It first selects task-relevant objects through Semantic Filtering (Section III-B), then derives semantic and physical constraints via Semantic Reasoning (Section III-C), and subsequently performs optimization-based Task Allocation (Section III-D). When a dynamic event occurs, this process is iteratively re-executed. The following subsections describe the 3DSG-based environmental grounding and each component of the Supervisor in detail through this example.

A. 3DSG-based Environment Grounding

In this work, we represent large-scale indoor environments as a hierarchical 3D Scene Graph (3DSG), defined as $G = (V, E)$, to enable environment-aware task planning. This representation is designed to serve as an interface between LLM-based reasoning and the optimization module. The node set $V = V_F \cup V_R \cup V_O$ consists of three semantic layers: floors (V_F), rooms (V_R), and objects (V_O), while the edge set E captures both hierarchical containment relationships and topological connectivity within the same layer.

G is provided in two formats: a NetworkX graph for path search and optimization, and a JSON structure for LLM-based constraint reasoning. An example of the room-level representation is shown below.

```
{
  "room_id": "A",
  "room_name": "hallway",
  "floor": "L1",
  "room_position": [...],
  "connected_rooms": ["meeting room", "stair
    "],
  "node_constraints": [],
  "edge_constraints": {
    "meeting room": [],
    "L1_stair": ["UAV only"]
  },
  "objects": [
    {
      "id": "fireextinguisher_1",
      "class": "fireextinguisher",
      "position": [...],
      "priority": "",
      "status": "pending"
    }
  ]
}
```

G is assumed to be fully constructed prior to execution. During runtime, the topological structure remains fixed, while object statuses, spatial restrictions (node and edge constraints), and task priorities are updated in response to dynamic events. This enables the LLM to consistently reference up-to-date environmental information without relying on conversation history, thereby reducing token overhead and ensuring logical consistency in replanning.

B. Semantic Filtering

Since G in large-scale indoor environments may contain hundreds of object nodes, passing the entire graph to the LLM is impractical due to token limitations. Furthermore, task-irrelevant objects introduce unnecessary noise into the constraint reasoning process, which can degrade the accuracy of allocation decisions.

To address this, the semantic filtering agent takes as input $\mathcal{I}$, dynamic events, and the full object list from G, and selects task-relevant object nodes $V_O^f \subseteq V_O$ through the commonsense reasoning capability of the LLM. The filtered graph is then constructed as $G_f = (V_f, E)$, where $V_f = V_F \cup V_R \cup V_O^f$, and serves as input to subsequent modules.

For example, given the initial instruction i_1: "Inspect all fire extinguishers in the building," only `fireextinguisher` nodes are selected into V_O^f. When an additional instruction i_2: "Also inspect fire hydrants" is issued, `firehydrant` nodes are added to V_O^f, and G_f is updated accordingly. This mechanism limits the context provided to the LLM to task-relevant objects, supporting consistent reasoning performance and scalability regardless of environment size.

C. Semantic Reasoning

The semantic reasoning agent takes as input $\mathcal{I}$, G_f, robot status information, and dynamic events. As summarized in Table I, the agent uses the LLM to categorize event-derived constraints into three primary types: task exclusion (C1), priority constraints (C2), and spatial restrictions (C3). Note that robot unavailability (C4) is detected directly by the Supervisor and handled without invoking this agent.

Task exclusion (C1) refers to the cancellation or removal of a specific task. **Priority constraint** (C2) is generated only when explicitly required by a dynamic event and represents an ordering constraint between tasks. **Spatial restriction** (C3) denotes an access restriction imposed on a specific node or edge, which is reflected in the `node_constraints` and `edge_constraints` fields of G. When a spatial restriction is imposed on a specific room node, tasks located in that room are automatically treated as task exclusions.

For example, a physical event such as "the corridor cannot be passed" is interpreted as a spatial restriction on the corresponding edge. The operator command "no entry to the meeting room" corresponds to both a spatial restriction on the room node and task exclusions for tasks located in that room. The instruction "inspect fire extinguishers before fire hydrants" is encoded as a priority constraint represented by a precedence relationship between tasks.

D. Task Allocation

The task allocation module takes as input the filtered graph G_f, the inferred constraints, and the skill and location information of each robot. It then uses an optimization solver to find an allocation that satisfies the given constraints while minimizing the overall travel cost.

Given a task set $\mathcal{T}$ and a robot set $\mathcal{R}$, priority constraints are reflected by partitioning $\mathcal{T}$ into ordered task stages, denoted as

$$\mathcal{T} = \mathcal{T}^{(1)} \cup \mathcal{T}^{(2)} \cup \cdots \cup \mathcal{T}^{(K)}, \tag{1}$$

where tasks in earlier stages have higher priority. For each stage $\mathcal{T}^{(k)}$, the binary assignment variable $x_{ij}^{(k)} \in \{0, 1\}$ is defined as 1 if robot i is assigned to task j in stage k, and 0 otherwise. The allocation problem for each stage is formulated as follows:

$$\text{minimize} \sum_{i \in \mathcal{R}} \sum_{j \in \mathcal{T}^{(k)}} c_{ij} x_{ij}^{(k)} \tag{2}$$

$$\sum_{i \in \mathcal{R}} x_{ij}^{(k)} = 1, \quad \forall j \in \mathcal{T}^{(k)} \tag{3}$$

$$x_{ij}^{(k)} = 0, \quad \text{if robot } i \text{ cannot perform task } j. \tag{4}$$

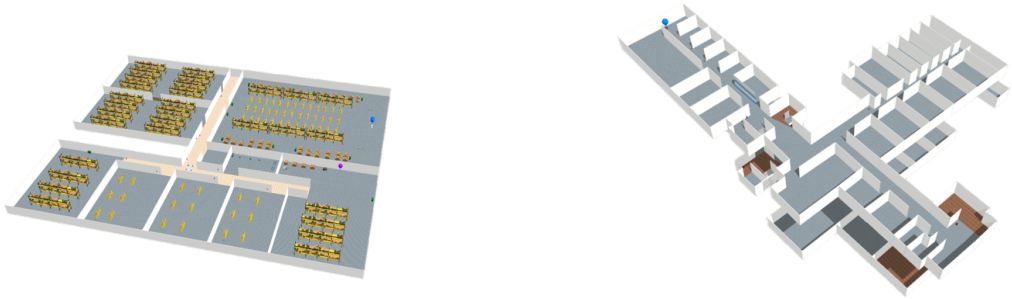
Here, c_{ij} denotes the travel cost for robot i to reach the waypoint of task j, computed based on the topological structure of G_f. Stages are solved sequentially from highest to lowest priority, allowing priority constraints inferred by the semantic reasoning agent to be enforced during allocation.

Spatial constraints are reflected by excluding inaccessible nodes or edges from path search and cost computation according to the `node_constraints` and `edge_constraints` fields of G_f. Task exclusions are handled by removing the corresponding tasks from $\mathcal{T}$. When a robot malfunction occurs, the affected robot is removed from $\mathcal{R}$ and the solver is immediately re-invoked to reallocate the remaining tasks. When other dynamic events occur, the allocation process is repeated using the updated G_f and inferred constraints, enabling timely task reallocation and replanning.

E. Supervisor

The Supervisor agent serves as the top-level module of EMTP and performs two key roles.

Online Adaptation Flow Manager. The Supervisor agent receives operator instructions and dynamic events in real time, classifies the event type, and controls the replanning pipeline by adaptively selecting and invoking the appropriate sub-agents. When a task addition is detected, the full replanning pipeline is executed starting from the Semantic Filtering agent. When a robot malfunction occurs, the Semantic Filtering and Semantic Reasoning agents are skipped, and the Task Allocation solver is immediately re-invoked for rapid reallocation. For events involving task exclusions, priority constraints, or spatial constraints, the Semantic Reasoning agent is invoked first to extract the relevant constraints, followed by Task Allocation.



(a) Environment A (b) Environment B

Fig. 2. Simulation environments used in the simulations. Environment A represents a large-scale single-floor warehouse environments, whereas Environment B represents a multi-floor indoor building environments.

TABLE I
CONSTRAINT TYPES AND REPRESENTATIVE EXAMPLES BY DIFFICULTY.

ID	Constraint	Detail	Examples by Difficulty
C1	Task Exclusion	Exclusion of tasks due to inaccessible space or operator instruction.	**Basic:** Do not inspect B1 and B2. **Paraphrase:** Do not inspect the fast-moving stocks. **Contextual:** The fire detectors in the main storage have already been inspected.
C2	Priority Constraint	Specification of task priority or dependency, typically based on operator instruction.	**Basic:** Inspect C1 before B1. **Paraphrase:** Inspect the fast-moving stocks before other areas. **Contextual:** Inspection of B1 depends on the stability of C1.
C3	Spatial Restriction	Room or corridor restriction due to environmental changes, mobility limits, or operator instruction.	**Basic:** R1 and R2 are not allowed to enter D2. **Paraphrase:** In the cold storage, no aerial robots are allowed. **Contextual:** The floor in D2 is covered in slippery oil.
C4	Robot Failure	Unavailability due to malfunction, communication loss, or operator instruction.	**Basic:** Malfunctions have been reported for Robot R1. **Paraphrase:** UAVs are non-operational. **Contextual:** The UAV has a broken propeller.

Output Verification and 3DSG Update. The Supervisor agent verifies the output of each sub-agent from two perspectives. First, it validates the output format and structure to ensure syntactic and formatting correctness. Second, it detects hallucinated references to objects or spaces that do not exist in the environmental information. When an error is detected, the output is corrected through post-processing rather than re-invoking the agent. Based on the verified results, the Supervisor updates G_f^t at each step t, maintaining the consistency of environmental information under dynamic conditions. Through these two roles, EMTP reliably combines the flexible reasoning capability of LLM-based agents with optimization-based task allocation.

IV. SIMULATIONS

A. Simulation Setup

All simulation experiments were conducted using OpenRMF, a ROS 2-based multi-fleet management middleware. The LLM orchestration pipeline was implemented using LangGraph with GPT-4o. Experiments were performed in two environments, as shown in Fig. 2.

- **Environment A:** A large-scale single-floor warehouse environment with a wide open layout and multiple rooms, supporting up to 100 tasks.
- **Environment B:** A two-floor indoor building environment with multiple rooms per floor, where inter-floor mobility constraints are inherently present, supporting up to 50 tasks.

Both environments use a baseline robot fleet consisting of two UGVs and one UAV. Environment A is used for both static and dynamic constraint experiments, while Environment B is used exclusively for dynamic replanning experiments to assess generalization across different spatial structures.

B. Constraint Handling Scenarios

To evaluate the planning efficiency and constraint satisfaction performance of the proposed EMTP, two scenarios are constructed. In both scenarios, the task instruction *"Inspect all fire safety equipment in the building"* is given. To assess the generalization performance of the system, various dynamic events are described in natural language, encompassing the difficulty levels and constraint types (C1–C4) listed in Table I.

a) Initial Planning under Static Constraints: At the start of each episode, one constraint group from C1–C4 is provided together with the task instruction, and the system generates and executes an initial plan that reflects the given constraints. No additional dynamic events occur during execution. This scenario evaluates planning efficiency and constraint satisfaction under static constraints, and is conducted in Environment

A, where the larger task scale better captures the effects of constraint handling on planning efficiency.

b) Online Replanning under Dynamic Constraints: The initial plan is generated from the task instruction alone, and new constraints are introduced sequentially during execution by predefined triggers or random events. Constraints accumulate over time, and each replanning step reflects all constraints introduced up to that point. During the replanning phase, all robots are temporarily paused. This scenario evaluates online replanning performance under dynamic and accumulating constraint changes, and is conducted in both Environment A and Environment B to assess generalization across environments with different spatial structures. To ensure a fair comparison of replanning performance, the same post-processing procedure was applied to all methods at each planning step to correct syntactic errors, invalid object references, and non-executable actions.

C. Comparative Methods

We evaluate EMTP against existing methods under identical input conditions, including the 3DSG, task descriptions, and constraints.

- **SMART-LLM** [21]: A method that sequentially performs task decomposition, team formation, and task allocation using LLMs, generating executable code at each stage.
- **LiP-LLM** [24]: A method that combines an LLM-generated dependency graph with linear programming to perform task allocation under precedence constraints.
- **COHERENT** [25]: A method that performs long-horizon task planning through a dialogue-based closed-loop pipeline consisting of plan, execute, feedback, and adapt stages.

D. Evaluation Metrics

Let $\mathcal{T}$ denote the set of all candidate tasks in an evaluation episode. For each run, $\mathcal{G} \subseteq \mathcal{T}$ denotes the ground-truth set of required tasks, $P \subseteq \mathcal{T}$ denotes the set of tasks actually executed, and $S \subseteq P$ denotes the subset of executed tasks completed without violating any constraints.

Success Rate (SR): A plan is considered successful (1) if it is parsable and consists only of valid object references present in the environment and executable robot actions. Otherwise, it is considered a failure (0).

Task Completion Rate (TCR): TCR is defined as the proportion of required tasks completed while satisfying all constraints:

$$\mathrm{TCR} = \frac{|\mathcal{G} \cap S|}{|\mathcal{G}|} \quad \text{(if SR = 1)}.$$

Constraint Satisfaction Rate (CSR): CSR is defined as the proportion of executed tasks completed while satisfying all constraints:

$$\mathrm{CSR} = \frac{|S|}{|P|} \quad \text{(if SR = 1)}.$$

Time & Token Usage: Planning time (s) and token usage (k tokens) are measured for each run.

TABLE II
EVALUATION RESULTS FOR INITIAL PLANNING UNDER STATIC CONSTRAINTS IN ENVIRONMENT A.

Const.	Method	SR↑	TCR↑	CSR↑	Time↓	Token↓
C1	**EMTP**	**1.00**	**1.00**	**0.93**	**12.1**	**12.38**
	SMART-LLM	1.00	0.79	0.89	42.19	32.1
	LiP-LLM	1.00	0.99	0.91	37.93	36.9
	COHERENT	1.00	0.89	0.92	23.47	25.5
C2	**EMTP**	**1.00**	**1.00**	**0.91**	**8.43**	**12.2**
	SMART-LLM	0.33	0.68	0.61	59.91	30.1
	LiP-LLM	0.71	0.92	0.59	114.13	42.4
	COHERENT	1.00	0.71	0.49	48.18	27.8
C3	**EMTP**	**1.00**	**0.91**	**0.78**	**10.58**	**12.3**
	SMART-LLM	0.89	0.74	0.77	38.85	32.4
	LiP-LLM	1.00	0.90	0.67	63.77	43.8
	COHERENT	1.00	0.74	0.62	31.94	28.0
C4	**EMTP**	**1.00**	**1.00**	**1.00**	**8.37**	**12.3**
	SMART-LLM	0.78	0.94	1.00	59.94	31.3
	LiP-LLM	1.00	0.70	0.67	43.77	37.0
	COHERENT	1.00	0.90	1.00	29.77	27.7

TABLE III
EVALUATION RESULTS FOR ONLINE REPLANNING UNDER DYNAMIC CONSTRAINTS.

Env.	Method	TCR↑	CSR↑	Time↓	Token↓
A	**EMTP**	**1.00**	**1.00**	**71.2**	**84.3**
	SMART-LLM	0.82	0.92	138.5	344.7
	LiP-LLM	0.90	0.90	258.4	295.6
	COHERENT	0.91	0.95	203.5	412.5
B	**EMTP**	**1.00**	**1.00**	**42.1**	**53.4**
	SMART-LLM	0.81	0.92	89.3	84.7
	LiP-LLM	0.58	0.58	148.3	103.3
	COHERENT	0.91	0.84	147.7	110.4

E. Results

As shown in Table II, EMTP achieved the most consistent planning performance across all constraint types, maintaining high TCR and CSR with substantially lower planning time and token usage than the baseline methods. SMART-LLM and COHERENT exhibited unstable performance across constraint types, as their LLM-driven pipelines frequently produced hallucinated or omitted tasks during parallel task processing. LiP-LLM achieved competitive performance in some cases, but showed a significant increase in planning time under priority constraints (C2), reflecting the scalability limitation of LLM-generated dependency graphs when complex precedence relationships are involved. In contrast, EMTP maintained stable task completion and constraint satisfaction through structured constraint handling and optimization-based allocation.

As shown in Table III, the advantage of EMTP became more pronounced in the online replanning scenario, where dynamic constraints were sequentially introduced and accumulated. To ensure a fair comparison of replanning performance, the same post-processing procedure was applied to all methods to correct syntactic errors, invalid object references, and non-

executable actions. Even with the common post-processing procedure, SMART-LLM and COHERENT showed noticeable degradation in TCR and CSR as interaction history and constraint information accumulated, often reassigning already completed tasks or ignoring newly introduced constraints during replanning. LiP-LLM also incurred increased planning time and token usage compared with the static scenario, reflecting the growing cost of repeatedly generating dependency structures under accumulated constraints. In contrast, EMTP maintained the highest TCR and CSR with the lowest planning cost in both environments, demonstrating stable and efficient replanning under dynamic conditions.

V. CONCLUSIONS

We have proposed EMTP, a framework for dynamic task allocation of heterogeneous multi-robot systems in large-scale indoor environments, in which 3DSG-based environment representation and hierarchical LLM orchestration are combined to handle complex constraints efficiently. The Supervisor's selective agent invocation minimizes unnecessary reasoning overhead, and the optimization-based task allocation ensures constraint-satisfying plans. Experiments confirm that the proposed method maintains low planning cost and high constraint satisfaction rates compared with existing methods, even under large task scales and accumulated dynamic constraints.

Several directions remain open. First, future work should improve the reliability of LLM-based output verification while reducing the additional computational cost and latency introduced by the verification process. Second, extending EMTP to real-world deployment requires further validation across diverse robot platforms and environment types. Addressing these issues will be important for deployment in more realistic multi-robot scenarios.

REFERENCES

[1] J. Liu, B. Yuan, H. Guan, H. Arkin, and R. Paul, "Grounding Complex Natural Language Commands for Temporal Tasks in Unseen Environments," in *Proc. Conf. Robot Learning (CoRL)*, 2023.

[2] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, "LLM+P: Empowering Large Language Models with Optimal Planning Proficiency," *arXiv preprint arXiv:2304.11477*, 2023.

[3] B. Joublin, A. Ceravola, P. Srikant, F. Otte, J. Deigmoeller, A. Belardinelli, C. Wang, F. Kyriazis, and C. Gienger, "AutoGPT+P: Affordance-based Task Planning with Large Language Models," in *Proc. Robotics: Science and Systems (RSS)*, 2024.

[4] K. Rana, M. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf, "SayPlan: Grounding Large Language Models using 3D Scene Graphs for Scalable Robot Task Planning," in *Proc. Conf. Robot Learning (CoRL)*, 2023.

[5] O. Kuempel, A. Z. Ren, D. Atha, K. Schmeckpeper, M. Pfeiffer, and A. Majumdar, "MoMa-LLM: Language-Grounded Dynamic Scene Graphs for Interactive Object Search With Mobile Manipulation," *IEEE Robot. Autom. Lett.*, 2024.

[6] Z. Yan, N. Jouandeau, and A. A. Cherif, "A Survey and Analysis of Multi-Robot Coordination," *Int. J. Adv. Robot. Syst.*, vol. 10, no. 12, p. 399, 2013.

[7] J. Motes, R. Sandstrom, H. Lee, S. Thomas, and N. M. Amato, "Multi-Robot Task and Motion Planning With Subtask Dependencies," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 3338–3345, Apr. 2020.

[8] S. Mayya, D. S. D'Antonio, D. Saldaña, and V. Kumar, "Resilient Task Allocation in Heterogeneous Multi-Robot Systems," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 1327–1334, Apr. 2021.

[9] G. Xu, Y. Wu, S. Tao, Y. Yang, T. Liu, T. Huang, H. Wu, and Y. Liu, "Efficient Multi-Robot Task and Path Planning in Large-Scale Cluttered Environments," *IEEE Robot. Autom. Lett.*, vol. 10, pp. 9112–9119, 2025, doi: 10.1109/LRA.2025.3592146.

[10] G. Neville, S. Chernova, and H. Ravichandar, "D-ITAGS: A Dynamic Interleaved Approach to Resilient Task Allocation, Scheduling, and Motion Planning," *IEEE Robot. Autom. Lett.*, vol. 8, no. 2, pp. 1037–1044, Feb. 2023.

[11] X. Luo and M. M. Zavlanos, "Temporal Logic Task Allocation in Heterogeneous Multirobot Systems," *IEEE Trans. Robot.*, vol. 38, no. 6, pp. 3602–3621, Dec. 2022.

[12] S. Nayak, M. W. Otte, S. Yeotikar, E. Carrillo, E. Rudnick-Cohen, M. K. M. Jaffar, R. Patel, S. Azarm, J. W. Herrmann, and H. Xu, "Experimental Comparison of Decentralized Task Allocation Algorithms Under Imperfect Communication," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 572–579, Apr. 2020.

[13] S. Wang, Y. Liu, Y. Qiu, and J. Zhou, "Consensus-Based Decentralized Task Allocation for Multi-Agent Systems and Simultaneous Multi-Agent Tasks," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 12593–12600, Oct. 2022.

[14] W. Dai, U. Rai, J. Chiun, Y. Cao, and G. Sartoretti, "Heterogeneous Multi-Robot Task Allocation and Scheduling via Reinforcement Learning," *IEEE Robot. Autom. Lett.*, vol. 10, no. 3, pp. 2654–2661, Mar. 2025, doi: 10.1109/LRA.2025.3534682.

[15] J. Banfi, A. Messing, C. Kroninger, E. Stump, S. Hutchinson, and N. Roy, "Hierarchical Planning for Heterogeneous Multi-Robot Routing Problems via Learned Subteam Performance," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 4464–4471, Apr. 2022.

[16] S. Paul, P. Ghassemi, and S. Chowdhury, "Learning Scalable Policies over Graphs for Multi-Robot Task Allocation using Capsule Attention Networks," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, pp. 8815–8822, 2022.

[17] S. Paul, W. Li, B. Smyth, Y. Chen, Y. Gel, and S. Chowdhury, "Efficient Planning of Multi-Robot Collective Transport using Graph Reinforcement Learning with Higher Order Topological Abstraction," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2023.

[18] W. Dai, A. Bidwai, and G. Sartoretti, "Dynamic Coalition Formation and Routing for Multirobot Task Allocation via Reinforcement Learning," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, pp. 1–7, 2024, doi: 10.1109/ICRA57147.2024.10611244.

[19] Z. Wang and M. Gombolay, "Learning Scheduling Policies for Multi-Robot Coordination With Graph Attention Networks," *IEEE Robot. Autom. Lett.*, vol. 5, no. 3, pp. 4509–4516, Jul. 2020.

[20] S. Ma, J. Ruan, Y. Du, R. Bucknall, and Y. Liu, "An End-to-End Deep Reinforcement Learning Based Modular Task Allocation Framework for Autonomous Mobile Systems," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 1519–1533, 2025.

[21] S. S. Kannan, V. L. N. Venkatesh, and B. C. Min, "SMART-LLM: Smart Multi-Agent Robot Task Planning using Large Language Models," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, pp. 12140–12147, Oct. 2024.

[22] Z. Mandi, S. Jain, and S. Song, "RoCo: Dialectic Multi-Robot Collaboration with Large Language Models," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, pp. 286–299, 2024.

[23] X. Zhang, H. Qin, F. Wang, Y. Dong, and J. Li, "LaMMA-P: Generalizable Multi-Agent Long-Horizon Task Allocation and Planning with LM-Driven PDDL Planner," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2025.

[24] K. Obata, T. Aoki, T. Horii, T. Taniguchi, and T. Nagai, "LiP-LLM: Integrating Linear Programming and Dependency Graph With Large Language Models for Multi-Robot Task Planning," *IEEE Robot. Autom. Lett.*, vol. 10, no. 2, pp. 1122–1129, Dec. 2024.

[25] K. Liu, Z. Tang, D. Wang, Z. Wang, B. Zhao, and X. Li, "COHERENT: Collaboration of Heterogeneous Multi-Robot System with Large Language Models," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, pp. 10208–10214, 2025.